

Cahier des Charges du projet [Réservation de Salle] Pour : TechSpace Solutions

1.Synopsis

Une application web permettant aux collaborateurs de TechSpace Solutions, de réserver la salle de réunion de manière autonome via un calendrier numérique et interactif.

Actuellement la gestion de salle est gérée via des post-it/mail/excel, ceci cause des problèmes de double réservation.

Utilisée par environ 50 employés sur ordinateurs pendant les heures de travail (8h-19h) du lundi au vendredi.

Ce projet sera livré aux clients dans un délai de 2 semaines.

2.Cahier des charges Client Front-end

Cahier des Charges Fonctionnel

L'interface du site doit permettre aux collaborateurs de TechSpace Solutions de gérer des réservations de salles de manière autonome et intuitive. L'objectif est de remplacer le système actuel par une application web centralisée (intranet) offrant une visibilité en temps réel sur la disponibilité de la salle.

USER STORIES

US-01 — Créer un compte (MUST)

En tant que visiteur,

Je veux créer un compte avec mon nom, prénom, email et mot de passe,

Afin de pouvoir accéder au système de réservation.

Critères d'acceptation :

- ☐ Le formulaire contient : email, mot de passe, confirmation du mot de passe, nom et prénom.
- ☐ Le mot de passe respecte une longueur minimale de 6 caractères.
- ☐ Les deux mots de passe saisis doivent correspondre.
- ☐ Si l'email existe déjà, un message d'erreur s'affiche.
- ☐ Redirection vers la page de connexion après succès.

US-02.1 - Se connecter (MUST) :

En tant que : Visiteur.

Je veux : m'authentifier.

Afin de : d'accéder au planning et à mon espace personnel.

Critères d'acceptation :

Le formulaire contient email + mot de passe. Message d'erreur si les identifiants sont incorrects. Après succès, accès au planning et stockage du JWT (cookie/localStorage).

US-02.2 - Se déconnecter (MUST) :

En tant que : Visiteur.

Je veux : me déconnecter.

Afin de : sécuriser mon compte et empêcher l'accès à mon espace personnel.

Critères d'acceptation :

Un bouton "Déconnexion" est visible lorsque l'utilisateur est authentifié. Au clic sur le bouton "Déconnexion" : Le JWT est supprimé du cookie (durée initiale : 7 jours). L'utilisateur est redirigé vers la page de connexion.

US-02.3 - Consulter son profil (MUST) :

En tant que : Utilisateur connecté.

Je veux : consulter mon profil.

Afin de : vérifier mes informations personnelles et les détails de mon compte.

Critères d'acceptation :

La page profil utilisateur est accessible uniquement lorsque l'utilisateur est connecté. Les informations personnelles sont visibles. Le profil affiche l'email et le nom (s'il existe). Les informations sont affichées en lecture seule (non modifiables).

US-03 - Voir le planning de la semaine (MUST):

En tant que : Utilisateur connecté.

Je veux : voir le planning de la semaine en cours.

Afin de : visualiser rapidement les créneaux libres et occupés.

Critères d'acceptation :

Affichage du lundi au vendredi, de 8h00 à 19h00. Distinction visuelle claire entre créneaux libres et occupés.

US-04.1 - Réserver un créneau (MUST):

En tant que : Utilisateur connecté.

Je veux : Réserver un créneau libre.

Afin d' : Organiser une réunion sans soucis.

Critères d'acceptation : Durée minimum de 1 heure. Pas de chevauchement possible avec une réservation existante. Impossible de réserver un créneau déjà passé. Saisie obligatoire de l'objet de la réunion.

US-04.2 - Voir qui a réservé un créneau (MUST NOT) :

En tant que : Utilisateur connecté.

Je veux : Voir si un créneau est réservé et par qui.

Afin de : Éviter les conflits de réservation et garantir un planning cohérent.

Critères d'acceptation : Les créneaux passés sont affichés en grisé et ne sont pas cliquables. Un créneau déjà réservé ne peut pas être réservé à nouveau. Lorsqu'un créneau est réservé, l'information de réservation est visible (ex. : réservé / nom ou identifiant utilisateur). Deux créneaux ne peuvent pas se chevaucher. Il est impossible de réserver un créneau dans le passé.

US-05 - Modifier ou annuler sa réservation (MUST):

En tant que : Utilisateur connecté.

Je veux : Modifier ou annuler ma propre réservation.

Afin de : Libérer la salle ou ajuster l'horaire si mon besoin change.

Critères d'acceptation : L'utilisateur ne peut modifier/annuler que ses propres réservations.

Mise à jour immédiate du planning après action.

US-06 - Réserver de façon flexible (COULD) :

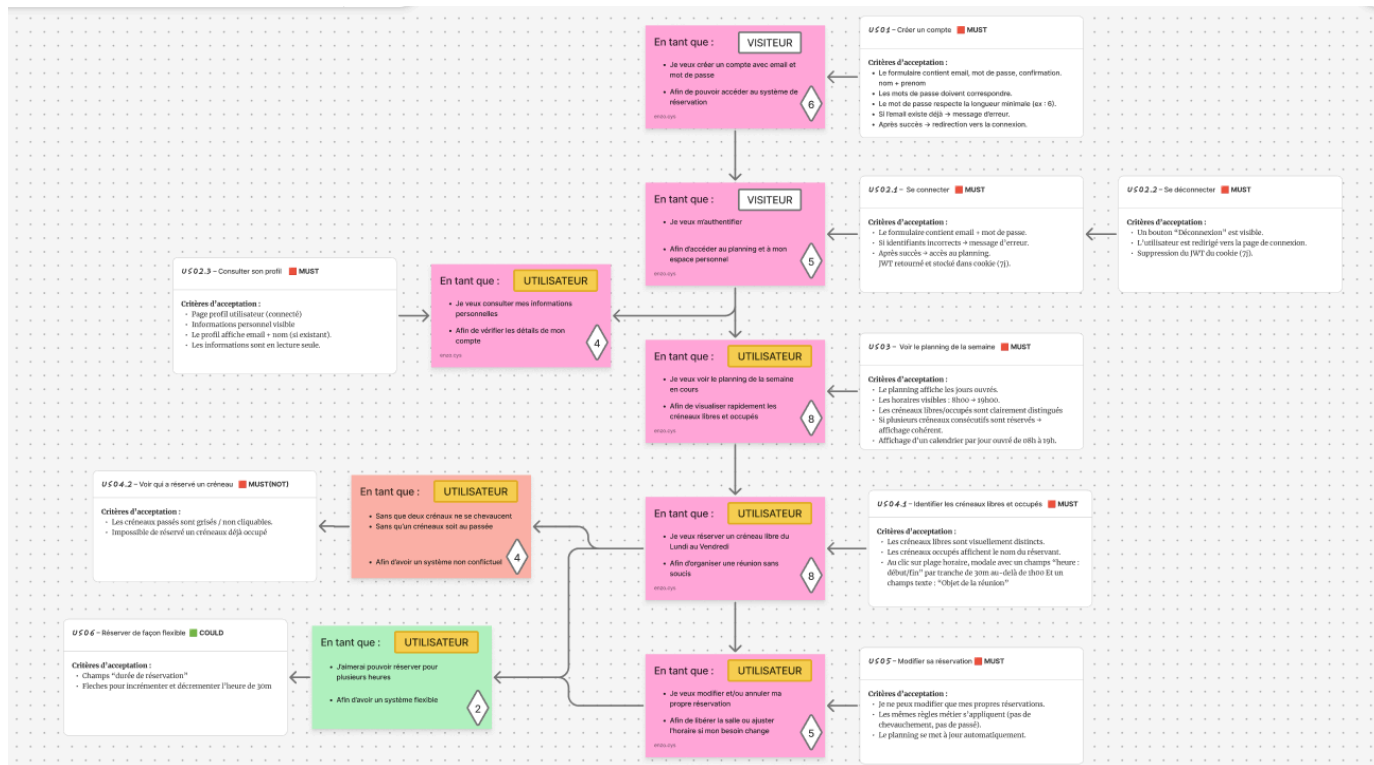
En tant que : Utilisateur.

Je veux : Pouvoir réserver un créneau pour une durée flexible.

Afin de : Bénéficier d'un système de réservation plus souple.

Critères d'acceptation : Un champ "durée de réservation" est disponible lors de la réservation. La durée est ajustable via des flèches d'incrément / décrémentation.

L'ajustement de la durée se fait par pas de 30 minutes. La durée sélectionnée met à jour automatiquement le créneau réservé.



...

Cahier des Charges Non-Fonctionnel

Ici est décrit la partie technique du front-end

Routes

<http://localhost:5000/> // Accueil, avec Lien vers Connexion OU Calendrier si Connecté
<http://localhost:5000/profil/:id> // Profil utilisateur
<http://localhost:5000/connexion> // Page Connexion
<http://localhost:5000/inscription> // Page Inscription

Contraintes de Sécurité :

Inscription et Connexion obligatoire pour l'ajout et la modification de réservation, Création et refresh du Token JWT (7j)

Inscription : mdp hashé, requête SQL préparer

calendrier : requête SQL préparer

Déploiement (devops,cicd,docker) :

Vercel

3.Cahier des charges client du Backend

Cahier des Charges Fonctionnel

Besoin :

Inscription en base de donnée des informations utilisateurs dans une table {USER} avec hachage du MDP via BCrypt

ID, Name, Surname, Email, Password, Updated_at

inscription en base de donnée des informations relatif à la réservation de salle dans une table {RESERVATION}

ID, Reunion Title, Organizer, Start Date, End Date, Created_at, Updated_at

anonymisation des données utilisateurs si suppression du compte

Cahier des Charges Non-Fonctionnel

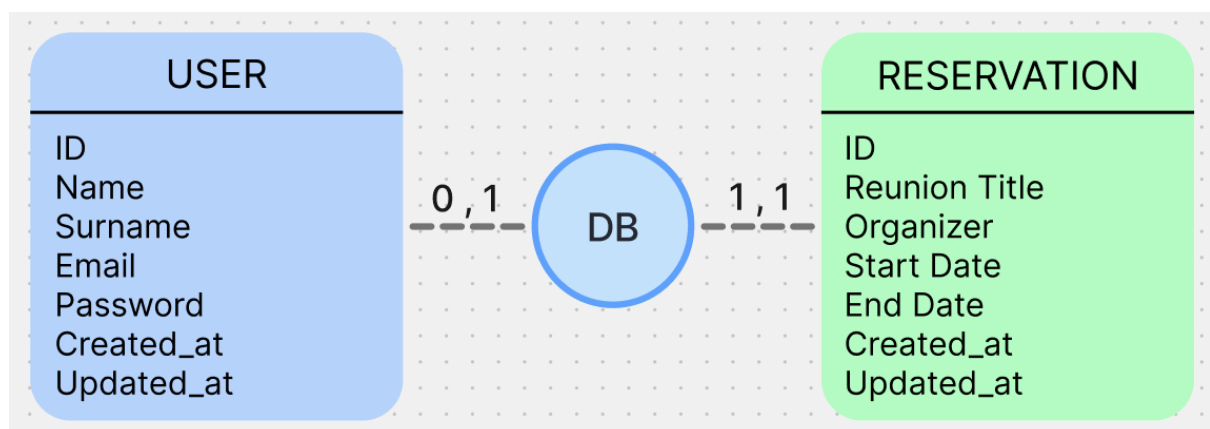
STACK :

M.E.R.N = Mysql, Express, React, NodeJS

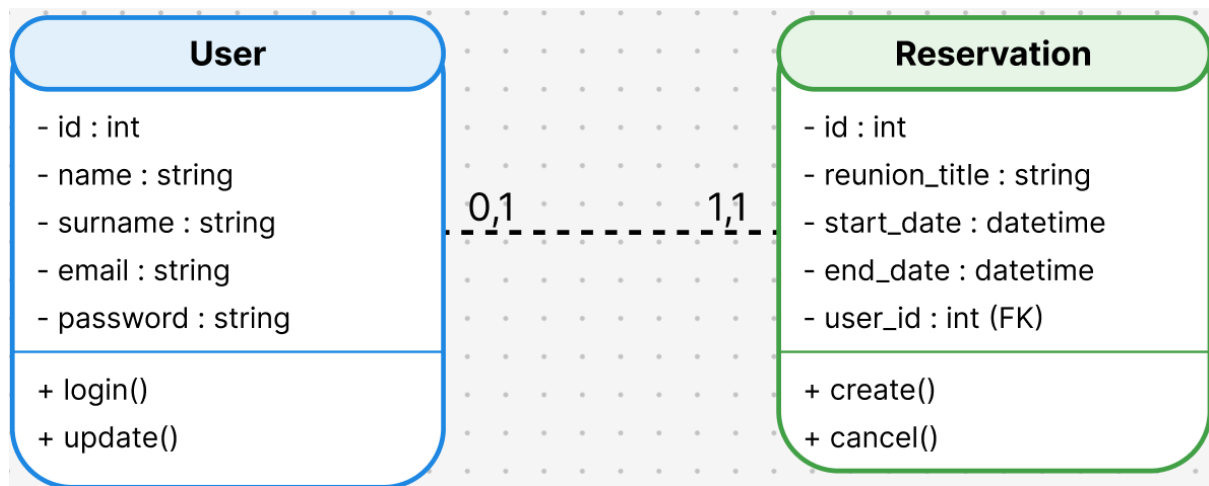
NodeJS, Express, MySQL2 = Backend

React, Vite, Tailwind CSS = FrontEnd

MCD :



Entité et Schéma UML entité relation



Commande SQL pour la DATABASE:

-- 1. Création de la base de données

```
CREATE DATABASE IF NOT EXISTS reservation_salles;
USE reservation_salles;
```

-- 2. Table des utilisateurs (Profil complet selon ton MCD)

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- 3. Table des réservations

```
CREATE TABLE reservations (
    id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(255) NOT NULL,
    start_date DATETIME NOT NULL,
    end_date DATETIME NOT NULL,
    user_id INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
-- Relation : Si l'utilisateur est supprimé, ses réservations le sont aussi
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

EndPoints

routes du back et leurs contraintes leurs code de status, leurs DTO :

<http://localhost:5000/Inscription>

Méthode Post, Code 201, Envoi :

```
{
  "name" : "gendrau",
  "surname" : "jean-elly",
  "email" : "jean-elly@test.com",
  "password" : "hashage"
}
```

Code 200, Recu :

```
{
  "ID" : "1",
  "name" : "gendrau",
  "surname" : "jean-elly",
  "email" : "jean-elly@test.com"
}
```

<http://localhost:5000/Connexion>

Méthode Post, Code 201, Envoi :

```
{
  "email" : "jean-elly@test.com",
  "password" : "hashage"
}
```

Code 200, Recu :

```
{
  "ID" : "1",
  "email" : "jean-elly@test.com"
}
```

<http://localhost:5000/>

Méthode Post, Code 201, Envoi :

```
{
  "reunion title" : "Reunion hebdomadaire",
  "organizer" : "jean-elly gendreau",
  "start date" : "15/02/2026 10:00",
  "end date" : "15/02/2026 11:00"
}
```

Code 200, Recu:

```
{
  "reunion title" : "Reunion hebdomadaire",
  "organizer" : "jean-elly gendreau",
  "start date" : "15/02/2026 10:00",
  "end date" : "15/02/2026 11:00"
}
```

<http://localhost:5000/profil/:id>

(anonymisation)

Méthode Post, Code 201, Envoi :

```
{
  "name" : "anonymisé",
  "surname" : "anonymisé",
  "email" : "anonymisé"
}
```

Méthode Post, Code 201, reçu:

```
{
  "name" : "anonymisé",
  "surname" : "anonymisé",
  "email" : "anonymisé"
}
```

Contraintes de Sécurité

- Authentification nécessaire pour envoyer des données en BDD via formulaire
- Authentification obligatoire pour avoir accès au planning
- clé JWT créer lors de l'inscription qui à la connexion entre dans les cookies avec une durée de 7j l'utilisateur devra se reconnecter pour re-générer un token JWT et re-avoir accès au planning
- Requêtes SQL préparer

RGDP

- Anonymisation : Bouton "supprimer le compte" dans le profil utilisateur, anonymisation en BDD

Déploiement (devops,cicd,docker)

Vercel

Logging

Niveau de logging : DEBUG, INFO, WARNING, ERROR de stdout

Code d'erreur

200 : Ok = profil utilisateur récupéré, planning récupéré...

201 : Created = Compte créé avec succès, réservation effectué

401 : Unauthorized = Token JWT invalide, créer un compte ou reconnectez-vous

404 : Not Found = Utilisateur inexistant (profil/***)

409 : Conflict = Email déjà utilisé

500 : Server Error = Connexion à la BDD impossible